

Pintos 4 정리

• 기존 Pintos file 시스템.

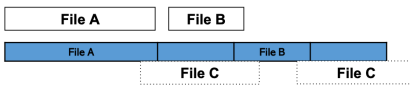
- 고정 크기 $\Rightarrow$ 생성시 file 크기 지정
- external fragmentation : 한 file의 sector들이 연속적으로 배치
 $\Rightarrow$ 공간 남아도 연속 공간이 아니면 file 배치 불가.
- EOF 이후 write 불가능

• Indexed and Extensible Files

- (파티션 크기 - 메타데이터) 만큼의 큰 파일을 지원해야 함.
- EOF에 쓰기 작업시 file 확장
- Sparse file 지원

filesystem/*.c
devices/block.c

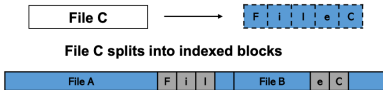
• Existing file system



- Cannot place file C although there is enough space

기존 : 연속공간 없음 X
 $\Rightarrow$ external fragmentation

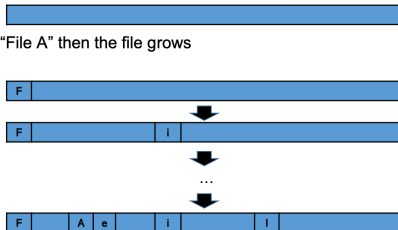
• Extensible file system



연속공간 아니어도 됨.

• File growth

- Open "File A" that did not exist before
- Write data to "File A" then the file grows



그럼 이해 안감

5.3.2 Indexed and Extensible Files 요약 정리

◆ 기존 파일 시스템 문제

- 단일 extent 기반 파일 저장 *start 기반 구조 삭제*
- 외부 단편화 문제 발생 *direct 기반으로 원상 저장*
- 파일 크기 고정 (생성 시 지정) *inode_write_at() 에서 inode_grow()*

◆ 해결 방안

- on-disk inode 구조 변경
- 인덱스 기반 구조 도입
 - direct block
 - indirect block
 - ✗ doubly indirect block (X) → 구현 안해도 통과
- 다른 방식 허용
 - 단, 디자인 문서에 타당성 설명 필수
 - external fragmentation 해결이 전제 조건

◆ 파일 크기 지원 범위

- 파일 시스템 파티션 $\leq 8MB$ *알기*
- 파일 최대 크기 $\approx 8MB$ - 메타데이터 크기 → doubly 이내서 안되긴 함.
- ✗ inode는 한 섹터(512B) 내에 위치
 - block pointer 수 제한 → doubly indirect 필수 (X) 안해도 통과

◆ 파일 성장 (file growth)

- indexed inode로 임의 위치 확장 가능
- write 시 자동 블록 할당 및 확장 *inode_write_at() → inode_grow()*
- seek 자체는 파일 확장 없음
- write past EOF → 파일 확장 + 0으로 채움(zero-fill gap)

◆ root directory 확장

- 기존 16개 제한 제거
 - 일반 파일과 동일하게 확장 가능
- 구현 막히긴 했지만 directory.c 보면*
dir_create() → inode_create 호출
dir_add() → inode_write_at() 호출 *하나씩 알아서 inode_grow도 확장됨.*

◆ sparse file 처리

- 쓰기 위치와 EOF 사이 gap 발생 가능
- zero block 처리 방식 선택 가능
 - 방식 1: 실제 디스크 블록 할당
 - 방식 2: 블록 미할당 (sparse file 방식) ← 안했는데 쿼리 나왔음 (Test는 통과)

◆ 구현 요구 사항 요약

- 파일 자동 확장 지원 ○
- indexed allocation 구조 도입 ○
- sparse file 처리 가능 ✗
- 파일 크기 상한 없음 (파일 시스템 전체 크기 이내) △
- seek/write/read 동작 일관성 유지 ○
- 루트 디렉토리 포함 확장 가능 ○

```

/// BLOCK_SECTOR_SIZE = 512 bytes에서 4 byte짜리 3개 빼고 남은 500 bytes를 direct_block에 할당 (4 bytes * 125 = 500 bytes)
#define INODE_DIRECT_CNT 125
/// BLOCK_SECTOR_SIZE = 512 bytes, block_sector_t = 4 bytes -> 한 블록에 sector 번호 128개 저장 가능
#define INDIRECT_BLOCK_CNT 128

/* On-disk inode.
| Must be exactly BLOCK_SECTOR_SIZE bytes long. */
You, 10시간 전 | 2 authors (You and one other)
struct inode_disk
{
    // block_sector_t start;                /* First data sector. */
    off_t length;                          /* File size in bytes. */  /// 4 bytes
    unsigned magic;                        /* Magic number. */    /// 4 bytes
    // uint32_t unused[125];                /* Not used. */
    /// direct 파일 데이터 블록 포인터
    block_sector_t direct_block[INODE_DIRECT_CNT];  /// 4 * 125 = 500 bytes
    /// indirect 참조 포인터 (간접 테이블로 블록들 연결)
    block_sector_t indirect_block_sector;  /// 4 bytes
};

```

```

static block_sector_t
byte_to_sector (const struct inode *inode, off_t pos)
{
    ASSERT (inode != NULL);
    /// 기존 방식
    // if (pos < inode->data.length)
    //     return inode->data.start + pos / BLOCK_SECTOR_SIZE;

    size_t idx = pos / BLOCK_SECTOR_SIZE;
    /// direct block
    if (idx < INODE_DIRECT_CNT)
        return inode->data.direct_block[idx]; // direct block에서 바로 반환

    /// TODO: indirect block
    if (idx < INODE_DIRECT_CNT + INDIRECT_BLOCK_CNT) {
        // indirect 블록 자체가 아직 할당 안됨
        if (inode->data.indirect_block_sector == 0)
            return -1;

        // indirect 블록 안에 있는 포인터 배열 읽어서 블록 포인터 찾기
        block_sector_t indirect_block[INDIRECT_BLOCK_CNT];
        block_read(fs_device, inode->data.indirect_block_sector, indirect_block);

        // indirect block 배열에서 해당 인덱스의 실제 data block 번호 return
        return indirect_block[idx - INODE_DIRECT_CNT];
    }
    else
        return -1;
}

```

```

bool
inode_create (block_sector_t sector, off_t length)
{
    struct inode_disk *disk_inode = NULL;
    bool success = false;

    ASSERT (length >= 0);

    /* If this assertion fails, the inode structure is not exactly
       | one sector in size, and you should fix that. */
    ASSERT (sizeof *disk_inode == BLOCK_SECTOR_SIZE);

    disk_inode = calloc (1, sizeof *disk_inode);
    if (disk_inode != NULL)
    {
        size_t sectors = bytes_to_sectors (length);
        disk_inode->length = length;
        disk_inode->magic = INODE_MAGIC;
        /// 기존 방식: start 색테로부터 연속적으로 할당하는 방식

        /// direct_block 배열에 하나씩 할당
        static char zeros[BLOCK_SECTOR_SIZE];
        size_t i;
        for (i = 0; i < sectors; i++) {
            if (i >= INODE_DIRECT_CNT) {
                break; // direct block 개수 초과 -> indirect block로
            }
            if (!free_map_allocate(1, &disk_inode->direct_block[i])) {
                return false;
            }
            block_write(fs_device, disk_inode->direct_block[i], zeros);
        }

        /// TODO: indirect_block
        if (sectors > INODE_DIRECT_CNT) {
            size_t indirect_sectors = sectors - INODE_DIRECT_CNT;

            // indirect block 안에 들어갈 색터 포인터 배열
            block_sector_t indirect_block[INDIRECT_BLOCK_CNT];
            memset(indirect_block, 0, sizeof indirect_block);

            // indirect block 자체에 해당하는 블록 한 개 할당
            if (!free_map_allocate(1, &disk_inode->indirect_block_sector)) {
                return false;
            }

            // indirect block 내 실제 데이터 블록들 할당
            size_t j;
            for (j = 0; j < indirect_sectors; j++, i++) {
                if (!free_map_allocate(1, &indirect_block[j])) {
                    return false;
                }
                // 0으로 초기화
                block_write(fs_device, indirect_block[j], zeros);
            }

            // indirect block sector에 포인터 배열 기록
            block_write(fs_device, disk_inode->indirect_block_sector, indirect_block);
        }

        // 전체 색터 정상 할당된 경우
        block_write(fs_device, sector, disk_inode);
    }
    return true;
}

```

```

bool inode_grow(struct inode *inode, off_t new_length) {
    // 기존파일과 신규 파일의 크기를 섹터 단위로 계산
    size_t old_sectors = bytes_to_sectors(inode->data.length);
    size_t new_sectors = bytes_to_sectors(new_length);

    // 0으로 채워진 메모리 버퍼 하나 생성
    static char zeros[BLOCK_SECTOR_SIZE];

    ///! direct block grow
    size_t i;
    for (i = old_sectors; i < new_sectors; i++) {
        if (i >= INODE_DIRECT_CNT) {
            break; // direct block 개수 초과 -> indirect block로
        }
        // 새로운 블록 할당 실패 -> grow 실패
        if (!free_map_allocate(1, &inode->data.direct_block[i])) {
            return false;
        }
        // 새로 할당된 블록을 0으로 초기화
        block_write(fs_device, inode->data.direct_block[i], zeros);
    }

    // TODO: indirect block grow
    if (new_sectors > INODE_DIRECT_CNT) {
        size_t indirect_needed = new_sectors - INODE_DIRECT_CNT;
        block_sector_t indirect_block[INDIRECT_BLOCK_CNT] = {0};

        // 기존 indirect block이 없다면 새로 할당
        if (inode->data.indirect_block_sector == 0) {
            if (!free_map_allocate(1, &inode->data.indirect_block_sector))
                return false;
        }
        else {
            block_read(fs_device, inode->data.indirect_block_sector, indirect_block);
        }

        // 기존에 indirect block까지 이미 일부 할당되어 있었는지 확인
        size_t indirect_start = 0;
        if (old_sectors > INODE_DIRECT_CNT){
            indirect_start = old_sectors - INODE_DIRECT_CNT;
        }

        for (i = indirect_start; i < indirect_needed; i++) {
            if (!free_map_allocate(1, &indirect_block[i]))
                return false;
            block_write(fs_device, indirect_block[i], zeros);
        }

        block_write(fs_device, inode->data.indirect_block_sector, indirect_block);
    }

    inode->data.length = new_length; // 길이 업데이트

    block_write(fs_device, inode->sector, &inode->data); // 디스크에 inode 정보 업데이트
    return true;
}

```

```

off_t
inode_write_at (struct inode *inode, const void *buffer_, off_t size,
                off_t offset)
{
    const uint8_t *buffer = buffer_;
    off_t bytes_written = 0;
    uint8_t *bounce = NULL;

    if (inode->deny_write_cnt)
        return 0;

    ///! grow 해야한다면
    off_t end = offset + size; // 쓰려는 마지막 바이트 위치
    if (end > inode->data.length) { /** 현재 파일 크기 (inode->data.length) 보다 더 뒤까지 쓰려고 하면
        if (!inode_grow(inode, end)) return 0; // grow 실패 -> 쓰기 실패
    }

```

호출